

Demo: From Bond Prices to Hull–White Future Yield Curves

A hands-on companion to Lecture 7

Table of contents

1	Goal	1
2	Setup	1
3	Step 1: Create observable bond-market data	2
4	Step 2: Bootstrap today's discount curve	4
4.1	Check the bootstrap by repricing every input bond	7
5	Step 3: Make the initial curve usable at any maturity	8
6	Step 4: Fit the Hull–White dynamics to option prices	8
6.1	Hull–White building blocks	9
6.2	Create artificial observable option quotes	10
6.3	Calibrate a and σ with a simple grid search	11
7	Step 5: Fit the time-dependent drift to today's curve	15
7.1	Verify that Hull–White reproduces today's discount curve	16
8	Step 6: Generate future yield curves	18
9	Step 7: Use a future curve to price a bond	21
10	Step 8: Plain-bond consistency at time 0	23
11	Checks and interpretation	24
12	Takeaways	25

1 Goal

This demo follows one complete fixed-income workflow:

1. start with **observable market prices** for coupon bonds;
2. bootstrap today's discount factors and zero-coupon yield curve;
3. calibrate Hull–White so that it exactly respects that curve;
4. fit the model's mean reversion and volatility to observable option prices;
5. generate yield curves at a future date in different interest-rate states; and
6. use those future curves to price remaining bond cash flows.

The data are artificial, so the demo is reproducible. Their magnitudes resemble prices and rates that could be observed in a government-bond market. All rates are continuously compounded unless stated otherwise.



The central idea

Today's curve determines today's prices. Hull–White adds a disciplined way to describe *future* curves. Calibration makes those two statements consistent.

2 Setup

We only need NumPy, pandas, and Matplotlib. The functions below use explicit inputs and return ordinary arrays or data frames so that each step is easy to inspect.

```
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from IPython.display import display

np.random.seed(411)

# A restrained, consistent plotting style.
BLUE = "#2C6EAA"
GOLD = "#D4A72C"
INK = "#263238"
LIGHT_BLUE = "#B9D6EC"
GRID = "#D9E1E8"
```

```
plt.rcParams.update({
    "figure.figsize": (8, 4.6),
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.alpha": 0.65,
    "grid.color": GRID,
    "axes.titleweight": "bold",
})
```

3 Step 1: Create observable bond-market data

Suppose the market contains annual-pay government bonds maturing in one through ten years. We first create a smooth but non-flat zero curve. This curve is hidden from the calibration routine; it is used only to manufacture internally consistent bond prices.

```
def hidden_zero_rate(maturity):
    """Return a realistic synthetic continuously compounded zero rate.

    Input
    -----
    maturity : number or NumPy array
        Time to maturity in years.

    Output
    -----
    number or NumPy array
        Continuously compounded annual zero rate, in decimal form.
    """
    maturity = np.asarray(maturity, dtype=float)
    return (
        0.031
        + 0.009 * np.exp(-maturity / 1.8) # initially inverted front end
        + 0.004 * (1 - np.exp(-maturity / 7.0)) # gentle long-end rise
    )

def price_annual_coupon_bond(coupon_rate, maturity, discount_factors, face=100.0):
    """Price an annual-pay bond from discount factors at integer years.
```

```

discount_factors[k - 1] must equal P(0, k).
"""

coupon = face * coupon_rate
cash_flows = np.full(maturity, coupon)
cash_flows[-1] += face
return float(np.sum(cash_flows * discount_factors[:maturity]))

maturities = np.arange(1, 11)
true_zero_rates = hidden_zero_rate(maturities)
true_discount_factors = np.exp(-true_zero_rates * maturities)

# Coupon rates are quoted market characteristics, not fitted model parameters.
coupon_rates = np.array([0.040, 0.038, 0.037, 0.036, 0.036,
                          0.037, 0.038, 0.039, 0.040, 0.041])

clean_prices = np.array([
    price_annual_coupon_bond(coupon, int(maturity), true_discount_factors)
    for coupon, maturity in zip(coupon_rates, maturities)
])

bond_market = pd.DataFrame({
    "maturity_years": maturities,
    "coupon_rate_pct": 100 * coupon_rates,
    "observed_clean_price": clean_prices,
})

bond_market["observed_clean_price"] = bond_market["observed_clean_price"].round(4)

display(bond_market)

```

	maturity_years	coupon_rate_pct	observed_clean_price
0	1	4.0	100.2528
1	2	3.8	100.4539
2	3	3.7	100.6346
3	4	3.6	100.6063
4	5	3.6	100.8010
5	6	3.7	101.4727
6	7	3.8	102.2558
7	8	3.9	103.1538
8	9	4.0	104.1689
9	10	4.1	105.3017

These are the quantities a trader can observe: maturity, coupon, and clean price. The zero rates used to manufacture the prices are not passed into the bootstrap. For simplicity, all valuation dates are coupon dates, accrued interest is zero, and each maturity has one suitable benchmark bond.

4 Step 2: Bootstrap today's discount curve

For the one-year bond, the only cash flow is coupon plus principal, so its price immediately gives $P(0,1)$. For each later bond, discount factors for earlier coupons are already known. The final discount factor is the only unknown:

$$P(0,n) = \frac{\text{Bond price} - \sum_{i=1}^{n-1} C P(0,i)}{100 + C}.$$

```
def bootstrap_discount_factors(maturities, coupon_rates, bond_prices, face=100.0):
    """Bootstrap annual discount factors from annual-pay coupon bonds.

    Inputs
    -----
    maturities : array-like of integers
        Consecutive maturities 1, 2, ..., N.
    coupon_rates : array-like
        Annual coupon rates in decimal form.
    bond_prices : array-like
        Observed clean prices per 100 of face value.

    Output
    -----
    NumPy array
        Discount factors P(0, 1), ..., P(0, N).
    """
    maturities = np.asarray(maturities, dtype=int)
    coupon_rates = np.asarray(coupon_rates, dtype=float)
    bond_prices = np.asarray(bond_prices, dtype=float)

    expected = np.arange(1, len(maturities) + 1)
    if not np.array_equal(maturities, expected):
        raise ValueError("Maturities must be consecutive integers starting at 1.")

    discount_factors = np.empty(len(maturities))
```

```

for row, maturity in enumerate(maturities):
    coupon = face * coupon_rates[row]

    # Present value of coupons before the maturity date.
    earlier_coupon_pv = coupon * discount_factors[:row].sum()

    # Solve the bond-pricing equation for the one remaining unknown.
    discount_factors[row] = (
        bond_prices[row] - earlier_coupon_pv
    ) / (face + coupon)

return discount_factors

bootstrapped_discounts = bootstrap_discount_factors(
    maturities,
    coupon_rates,
    clean_prices,
)
bootstrapped_zero_rates = -np.log(bootstrapped_discounts) / maturities

curve_table = pd.DataFrame({
    "maturity_years": maturities,
    "discount_factor": bootstrapped_discounts,
    "zero_rate_pct": 100 * bootstrapped_zero_rates,
})
display(curve_table.round(6))

```

	maturity_years	discount_factor	zero_rate_pct
0	1	0.9640	3.6696
1	2	0.9325	3.4957
2	3	0.9028	3.4094
3	4	0.8738	3.3716
4	5	0.8453	3.3601
5	6	0.8173	3.3624
6	7	0.7898	3.3713
7	8	0.7629	3.3830
8	9	0.7367	3.3955
9	10	0.7112	3.4076

```

fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.1))

axes[0].plot(maturities, bootstrapped_discounts, "o-", color=BLUE)
axes[0].set(title="Bootstrapped discount factors",
            xlabel="Maturity (years)", ylabel="$P(0,T)$")

axes[1].plot(maturities, 100 * bootstrapped_zero_rates, "o-", color=BLUE)
axes[1].set(title="Bootstrapped zero-coupon curve",
            xlabel="Maturity (years)", ylabel="Continuous zero rate (%)")

plt.tight_layout()
plt.show()

```

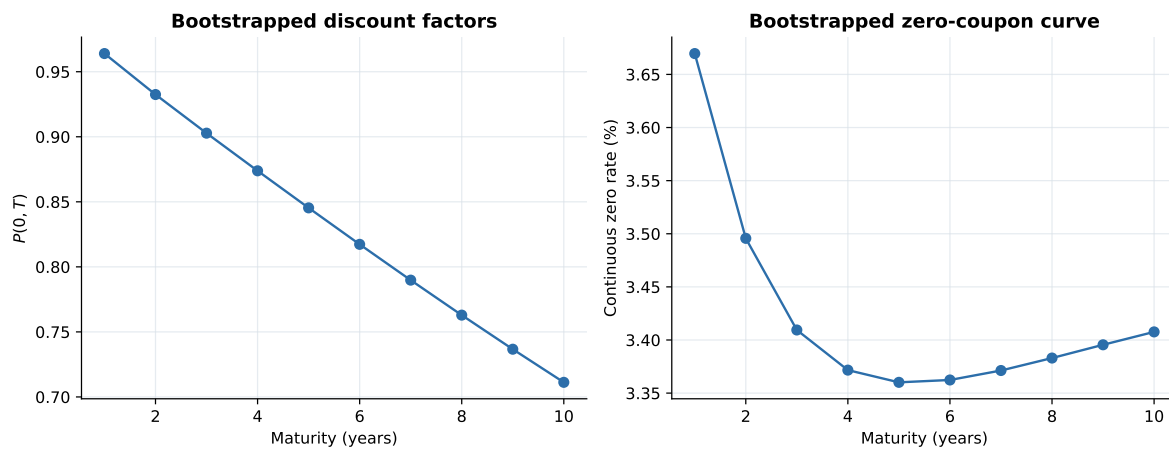


Figure 1: Today's curve recovered from observable coupon-bond prices

4.1 Check the bootstrap by repricing every input bond

A useful calibration habit is to return to the original observable quantities.

```

repriced_bonds = np.array([
    price_annual_coupon_bond(coupon, int(maturity), bootstrapped_discounts)
    for coupon, maturity in zip(coupon_rates, maturities)
])

bootstrap_check = bond_market.copy()
bootstrap_check["repriced_from_curve"] = repriced_bonds
bootstrap_check["pricing_error"] = repriced_bonds - clean_prices

```

```
display(bootstrap_check.round(8))

print(f"Largest absolute repricing error: "
      f"{np.max(np.abs(repriced_bonds - clean_prices)):.10f} per 100 face")
```

	maturity_years	coupon_rate_pct	observed_clean_price	repriced_from_curve	pricing_error
0	1	4.0	100.2528	100.2528	0.0
1	2	3.8	100.4539	100.4539	0.0
2	3	3.7	100.6346	100.6346	0.0
3	4	3.6	100.6063	100.6063	0.0
4	5	3.6	100.8010	100.8010	0.0
5	6	3.7	101.4727	101.4727	0.0
6	7	3.8	102.2558	102.2558	0.0
7	8	3.9	103.1538	103.1538	0.0
8	9	4.0	104.1689	104.1689	0.0
9	10	4.1	105.3017	105.3017	0.0

Largest absolute repricing error: 0.0000000000 per 100 face

5 Step 3: Make the initial curve usable at any maturity

Hull–White formulas require $P(0, T)$ and the instantaneous forward rate $f(0, t)$. We use linear interpolation of the continuously compounded zero rate. This is intentionally simple; production systems use richer curve construction.

```
def zero_rate_0(maturity):
    """Interpolate today's bootstrapped continuous zero rate."""
    maturity = np.asarray(maturity, dtype=float)
    return np.interp(
        maturity,
        np.r_[0.0, maturities],
        np.r_[bootstrapped_zero_rates[0], bootstrapped_zero_rates],
    )

def discount_0(maturity):
    """Return today's discount factor  $P(0, T)$ ."""
    maturity = np.asarray(maturity, dtype=float)
    return np.exp(-zero_rate_0(maturity) * maturity)
```

```
def forward_rate_0(time, step=1e-4):
    """Approximate  $f(0,t) = -d \log(P(0,t)) / dt$  numerically."""
    time = np.asarray(time, dtype=float)
    left = np.maximum(time - step, 0.0)
    right = time + step
    return -(np.log(discount_0(right)) - np.log(discount_0(left))) / (right - left)
```

6 Step 4: Fit the Hull–White dynamics to option prices

The lecture’s calibration logic separates two jobs:

- the function $\theta(t)$ makes the model fit today’s *entire curve*; and
- a and σ control future mean reversion and volatility.

In practice, a and σ are fitted to liquid caps, floors, or swaptions. To keep the code transparent, our artificial option market consists of European calls on zero-coupon bonds. Such option premiums are observable, and Hull–White has a short closed-form pricing formula for them.

6.1 Hull–White building blocks

For the one-factor model

$$dr_t = [\theta(t) - ar_t]dt + \sigma dW_t,$$

define

$$B(t, T) = \frac{1 - e^{-a(T-t)}}{a}.$$

The volatility of a zero-coupon bond option expiring at S on a bond maturing at T is

$$\sigma_P = \sigma B(S, T) \sqrt{\frac{1 - e^{-2aS}}{2a}}.$$

```

def normal_cdf(x):
    """Standard normal cumulative probability; works for scalars."""
    return 0.5 * (1.0 + math.erf(x / math.sqrt(2.0)))

def hw_B(start, end, mean_reversion):
    """Return the Hull-White B(start, end) loading."""
    return (1.0 - np.exp(-mean_reversion * (end - start))) / mean_reversion

def zero_bond_call_price(expiry, bond_maturity, strike,
                        mean_reversion, volatility, face=100.0):
    """Price a European call on a zero-coupon bond under Hull-White.

    The option expires at `expiry`. Its underlying pays `face` at
    `bond_maturity`. The strike is paid at expiry per `face` of principal.
    """
    p_0_s = float(discount_0(expiry))
    p_0_t = float(discount_0(bond_maturity))

    bond_vol = (
        volatility
        * hw_B(expiry, bond_maturity, mean_reversion)
        * math.sqrt((1.0 - math.exp(-2.0 * mean_reversion * expiry))
                    / (2.0 * mean_reversion))
    )

    # Work with values per $1 face, then multiply by face at the end.
    strike_per_dollar = strike / face
    h = (
        math.log(p_0_t / (strike_per_dollar * p_0_s)) / bond_vol
        + 0.5 * bond_vol
    )
    call_per_dollar = (
        p_0_t * normal_cdf(h)
        - strike_per_dollar * p_0_s * normal_cdf(h - bond_vol)
    )
    return face * call_per_dollar

```

6.2 Create artificial observable option quotes

We create at-the-money-forward calls with expiries from six months to five years. The hidden values $a = 0.18$ and $\sigma = 1.20\%$ are used only to generate the market premiums. A small alternating bid/ask-like perturbation prevents a mechanically perfect parameter recovery.

```
option_expiries = np.array([0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 5.0])
underlying_maturities = option_expiries + 5.0

# An ATM-forward strike makes the option comparison especially clear.
option_strikes = np.array([
    100.0 * discount_0(maturity) / discount_0(expiry)
    for expiry, maturity in zip(option_expiries, underlying_maturities)
])

hidden_a = 0.18
hidden_sigma = 0.012
quote_perturbation = np.array([1.003, 0.998, 1.002, 0.997, 1.001, 0.999, 1.002])

observed_option_prices = np.array([
    zero_bond_call_price(expiry, maturity, strike, hidden_a, hidden_sigma)
    for expiry, maturity, strike in zip(
        option_expiries, underlying_maturities, option_strikes
    )
]) * quote_perturbation

option_market = pd.DataFrame({
    "option_expiry": option_expiries,
    "bond_maturity": underlying_maturities,
    "strike_per_100": option_strikes,
    "observed_option_price": observed_option_prices,
})
display(option_market.round(5))
```

	option_expiry	bond_maturity	strike_per_100	observed_option_price
0	0.5	5.5	84.6605	0.8901
1	1.0	6.0	84.7855	1.1797
2	1.5	6.5	84.7806	1.3678
3	2.0	7.0	84.6983	1.4838
4	3.0	8.0	84.5051	1.6322
5	4.0	9.0	84.3051	1.6909

	option_expiry	bond_maturity	strike_per_100	observed_option_price
6	5.0	10.0	84.1344	1.7124

6.3 Calibrate a and σ with a simple grid search

The objective is the root-mean-square pricing error across the option quotes. A grid search is slower than a numerical optimizer, but students can see exactly what is tried and it needs no extra package.

```
def option_rmse(mean_reversion, volatility):
    """Return RMSE between Hull-White and observed option prices."""
    model_prices = np.array([
        zero_bond_call_price(expiry, maturity, strike,
                             mean_reversion, volatility)
        for expiry, maturity, strike in zip(
            option_expiries, underlying_maturities, option_strikes
        )
    ])
    return float(np.sqrt(np.mean((model_prices - observed_option_prices) ** 2)))

a_grid = np.linspace(0.03, 0.40, 150)
sigma_grid = np.linspace(0.006, 0.020, 150)

# Each matrix cell is the pricing error for one candidate (a, sigma) pair.
error_surface = np.array([
    [option_rmse(a, sigma) for sigma in sigma_grid]
    for a in a_grid
])

best_row, best_col = np.unravel_index(np.argmin(error_surface), error_surface.shape)
calibrated_a = a_grid[best_row]
calibrated_sigma = sigma_grid[best_col]

print(f"Calibrated mean reversion a: {calibrated_a:.4f} per year")
print(f"Calibrated volatility sigma: {100 * calibrated_sigma:.4f}% per sqrt(year)")
print(f"Option-price RMSE: {error_surface[best_row, best_col]:.6f} per 100 face")
```

```
Calibrated mean reversion a: 0.1765 per year
Calibrated volatility sigma: 1.1826% per sqrt(year)
Option-price RMSE: 0.005885 per 100 face
```

```

fig, ax = plt.subplots(figsize=(8, 4.8))
contour = ax.contourf(
    100 * sigma_grid,
    a_grid,
    np.log10(error_surface + 1e-8),
    levels=25,
    cmap="Blues_r",
)
ax.scatter(100 * calibrated_sigma, calibrated_a, s=90, color=GOLD,
           edgecolor=INK, label="Best grid point", zorder=3)
ax.set(xlabel="$\sigma$ (% per $\sqrt{\mathrm{year}}$)",
       ylabel="$a$ (per year)",
       title="Hull-White calibration objective")
ax.legend()
fig.colorbar(contour, ax=ax, label="log10(option-price RMSE)")
plt.show()

```

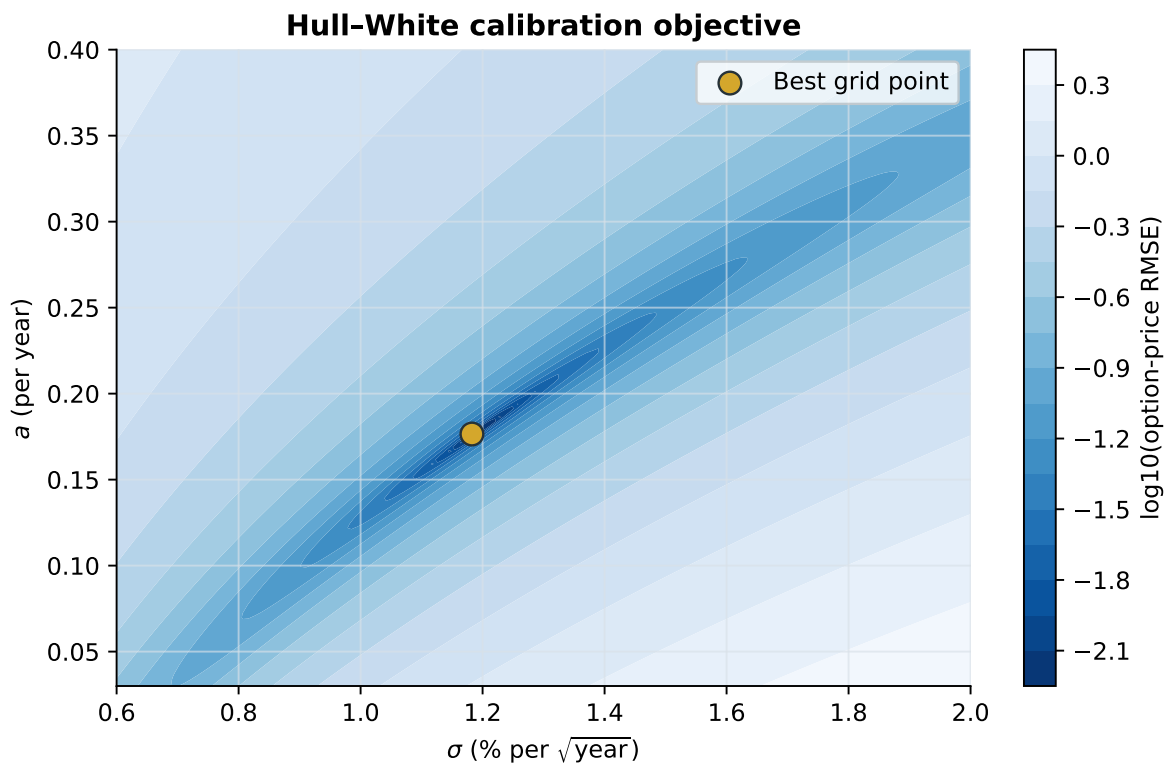


Figure 2: Option-pricing error across candidate Hull–White parameters

```

fitted_option_prices = np.array([
    zero_bond_call_price(expiry, maturity, strike,
                        calibrated_a, calibrated_sigma)
    for expiry, maturity, strike in zip(
        option_expiries, underlying_maturities, option_strikes
    )
])

option_fit = option_market.copy()
option_fit["model_price"] = fitted_option_prices
option_fit["pricing_error"] = fitted_option_prices - observed_option_prices
display(option_fit.round(6))

```

	option_expiry	bond_maturity	strike_per_100	observed_option_price	model_price	pricing_error
0	0.5	5.5	84.6605	0.8901	0.8818	-0.0083
1	1.0	6.0	84.7855	1.1797	1.1755	-0.0042
2	1.5	6.5	84.7806	1.3678	1.3585	-0.0093
3	2.0	7.0	84.6983	1.4838	1.4821	-0.0017
4	3.0	8.0	84.5051	1.6322	1.6259	-0.0063
5	4.0	9.0	84.3051	1.6909	1.6895	-0.0014
6	5.0	10.0	84.1344	1.7124	1.7074	-0.0050

```

plt.plot(option_expiries, observed_option_prices, "o-", color=BLUE,
        label="Observed option premiums")
plt.plot(option_expiries, fitted_option_prices, "s--", color=GOLD,
        label="Hull-White fitted premiums")
plt.xlabel("Option expiry (years)")
plt.ylabel("Option price per 100 face")
plt.title("Fit to the option market")
plt.legend()
plt.show()

```

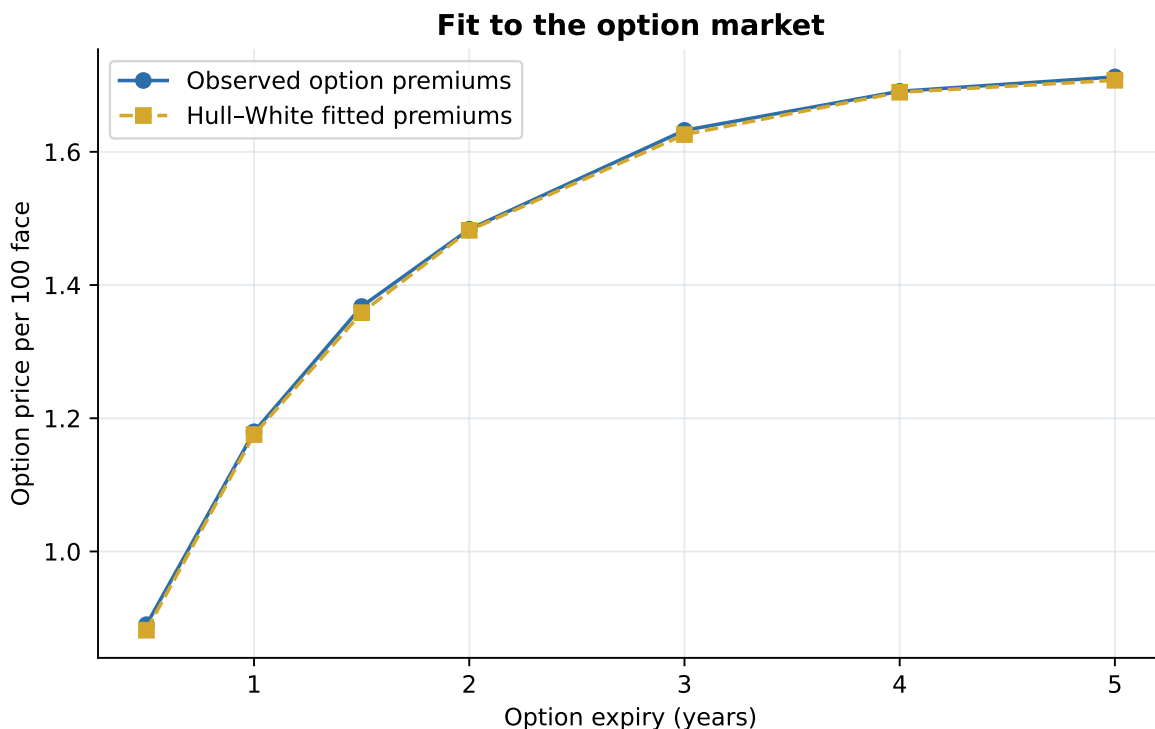


Figure 3: Observed artificial option premiums and calibrated Hull–White prices

7 Step 5: Fit the time-dependent drift to today's curve

Once a and σ are known, the drift function that fits the observed initial curve is

$$\theta(t) = \frac{\partial f(0, t)}{\partial t} + af(0, t) + \frac{\sigma^2}{2a} (1 - e^{-2at}).$$

This is the lecture's key calibration point: $\theta(t)$ is a *function*, not one number. It supplies enough flexibility to match all maturities of today's curve.

```
def theta(time, mean_reversion, volatility, step=1e-3):
    """Return the Hull-White drift theta(t) fitted to today's curve."""
    time = np.asarray(time, dtype=float)
    left = np.maximum(time - step, 0.0)
    right = time + step

    # Numerical derivative of today's instantaneous forward curve.
```

```

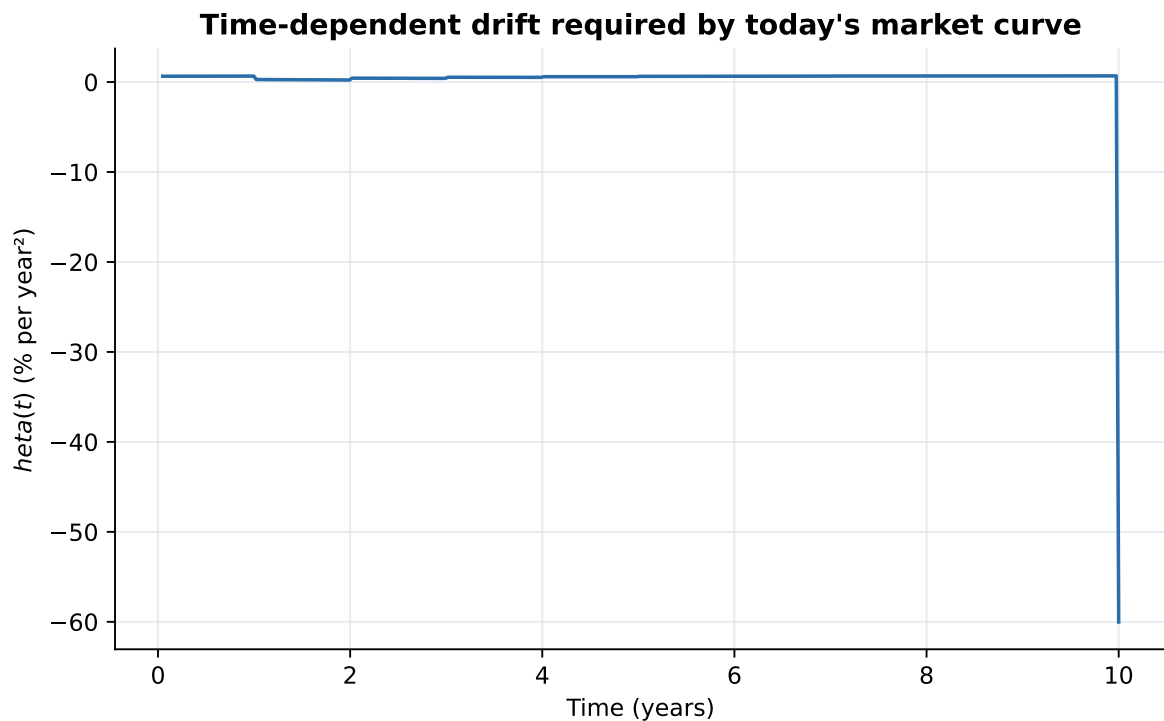
forward_slope = (
    forward_rate_0(right) - forward_rate_0(left)
) / (right - left)

return (
    forward_slope
    + mean_reversion * forward_rate_0(time)
    + volatility**2 / (2.0 * mean_reversion)
      * (1.0 - np.exp(-2.0 * mean_reversion * time))
)

theta_times = np.linspace(0.05, 10.0, 400)
theta_values = theta(theta_times, calibrated_a, calibrated_sigma)

plt.plot(theta_times, 100 * theta_values, color=BLUE)
plt.xlabel("Time (years)")
plt.ylabel("$\theta(t)$ (% per year2)")
plt.title("Time-dependent drift required by today's market curve")
plt.show()

```



The sharp changes are a consequence of using simple piecewise-linear interpolation. That is instructive: curve construction and smoothing matter in real calibration.

7.1 Verify that Hull–White reproduces today’s discount curve

For future-date pricing, Hull–White gives the zero-coupon bond price

$$P(t, T) = A(t, T)e^{-B(t, T)r_t},$$

where

$$A(t, T) = \frac{P(0, T)}{P(0, t)} \exp \left[B(t, T)f(0, t) - \frac{\sigma^2}{4a}(1 - e^{-2at})B(t, T)^2 \right].$$

At $t = 0$, using $r_0 = f(0, 0)$ makes the exponential terms cancel, leaving exactly $P(0, T)$. This is the no-arbitrage fit.

```
def hw_discount_factor(valuation_time, maturity, short_rate,
                       mean_reversion, volatility):
    """Return the Hull-White state-dependent discount factor P(t,T).

    Inputs
    -----
    valuation_time : float
        Future valuation date t, in years from today.
    maturity : number or array
        Payment date(s) T, with T > t.
    short_rate : float
        Short rate r_t observed or simulated at the valuation date.
    mean_reversion, volatility : float
        Calibrated Hull-White parameters a and sigma.
    """
    maturity = np.asarray(maturity, dtype=float)
    b_value = hw_B(valuation_time, maturity, mean_reversion)

    variance_adjustment = (
        volatility**2
        / (4.0 * mean_reversion)
        * (1.0 - np.exp(-2.0 * mean_reversion * valuation_time))
        * b_value**2
    )
```

```

a_value = (
    discount_0(maturity) / discount_0(valuation_time)
    * np.exp(
        b_value * forward_rate_0(valuation_time)
        - variance_adjustment
    )
)
return a_value * np.exp(-b_value * short_rate)

r_0 = float(forward_rate_0(0.0))
hw_discounts_today = hw_discount_factor(
    0.0, maturities, r_0, calibrated_a, calibrated_sigma
)

initial_fit = pd.DataFrame({
    "maturity": maturities,
    "market_discount": bootstrapped_discounts,
    "hw_discount": hw_discounts_today,
    "difference": hw_discounts_today - bootstrapped_discounts,
})
display(initial_fit.round(10))

```

	maturity	market_discount	hw_discount	difference
0	1	0.9640	0.9640	-0.0
1	2	0.9325	0.9325	0.0
2	3	0.9028	0.9028	0.0
3	4	0.8738	0.8738	0.0
4	5	0.8453	0.8453	0.0
5	6	0.8173	0.8173	0.0
6	7	0.7898	0.7898	0.0
7	8	0.7629	0.7629	-0.0
8	9	0.7367	0.7367	0.0
9	10	0.7112	0.7112	0.0

8 Step 6: Generate future yield curves

Move to year 2. In a one-factor Hull-White model, the short rate r_2 is the single state variable. Once a state is specified, `hw_discount_factor()` produces $P(2, T)$ for every $T > 2$,

and therefore the entire future zero curve:

$$y(2, T) = -\frac{\log P(2, T)}{T - 2}.$$

We show three possible states around the model's central short-rate level. They are scenarios, not point forecasts.

```
future_time = 2.0
future_maturities = np.linspace(2.25, 10.0, 80)
central_short_rate = float(forward_rate_0(future_time))

short_rate_states = {
    "Low-rate state": central_short_rate - 0.015,
    "Central state": central_short_rate,
    "High-rate state": central_short_rate + 0.015,
}

future_curve_rows = []
for state_name, short_rate in short_rate_states.items():
    state_discounts = hw_discount_factor(
        future_time,
        future_maturities,
        short_rate,
        calibrated_a,
        calibrated_sigma,
    )
    state_yields = -np.log(state_discounts) / (future_maturities - future_time)

    future_curve_rows.append(pd.DataFrame({
        "state": state_name,
        "short_rate": short_rate,
        "maturity": future_maturities,
        "zero_rate": state_yields,
    }))

future_curves = pd.concat(future_curve_rows, ignore_index=True)
display(future_curves.groupby("state").head(3).round(5))
```

	state	short_rate	maturity	zero_rate
0	Low-rate state	0.0174	2.2500	0.0184

	state	short_rate	maturity	zero_rate
1	Low-rate state	0.0174	2.3481	0.0184
2	Low-rate state	0.0174	2.4462	0.0185
80	Central state	0.0323	2.2500	0.0330
81	Central state	0.0323	2.3481	0.0330
82	Central state	0.0323	2.4462	0.0329
160	High-rate state	0.0474	2.2500	0.0477
161	High-rate state	0.0474	2.3481	0.0475
162	High-rate state	0.0474	2.4462	0.0473

```

styles = {
    "Low-rate state": (BLUE, "-"),
    "Central state": (INK, "--"),
    "High-rate state": (GOLD, "-."),
}

for state_name, group in future_curves.groupby("state", sort=False):
    color, line_style = styles[state_name]
    plt.plot(group["maturity"] - future_time, 100 * group["zero_rate"],
             color=color, linestyle=line_style, linewidth=2,
             label=f"{state_name} (r = {100 * group['short_rate'].iloc[0]:.2f}%)")

plt.xlabel("Remaining maturity, $T-2$ (years)")
plt.ylabel("Year-2 zero rate (%)")
plt.title("Hull-White yield curves at a future date")
plt.legend()
plt.show()

```

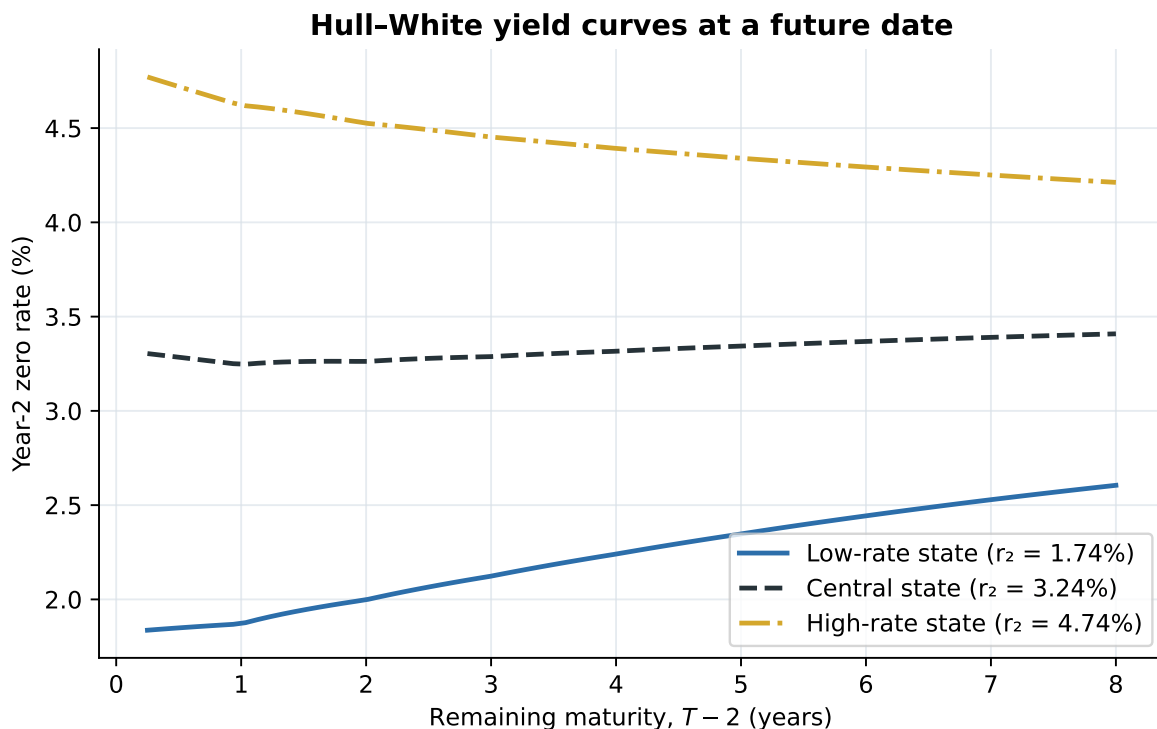


Figure 4: At year 2, one short-rate state determines a complete remaining yield curve

Notice that one factor moves the whole curve in a structured way. Mean reversion causes the effect of today's short-rate state to weaken at longer maturities because $B(t, T)$ approaches $1/a$ rather than increasing without bound.

9 Step 7: Use a future curve to price a bond

Consider the original ten-year 4.10% coupon bond at year 2, immediately after its year-2 coupon has been paid. Eight coupons and the principal remain. Its value in each state is simply the discounted value of those remaining cash flows using $P(2, T)$.

```
def price_bond_at_time(valuation_time, payment_times, cash_flows,
                      short_rate, mean_reversion, volatility):
    """Price fixed cash flows at a future date using a Hull-White curve.

    Output is the value at `valuation_time`, not today's value.
    """
    payment_times = np.asarray(payment_times, dtype=float)
```

```

cash_flows = np.asarray(cash_flows, dtype=float)

if np.any(payment_times <= valuation_time):
    raise ValueError("Every payment must occur after the valuation time.")

future_discounts = hw_discount_factor(
    valuation_time,
    payment_times,
    short_rate,
    mean_reversion,
    volatility,
)
return float(np.sum(cash_flows * future_discounts))

payment_times = np.arange(3.0, 11.0)
remaining_cash_flows = np.full(len(payment_times), 4.10)
remaining_cash_flows[-1] += 100.0

future_bond_values = []
for state_name, short_rate in short_rate_states.items():
    value = price_bond_at_time(
        future_time,
        payment_times,
        remaining_cash_flows,
        short_rate,
        calibrated_a,
        calibrated_sigma,
    )
    future_bond_values.append({
        "state": state_name,
        "short_rate_pct": 100 * short_rate,
        "bond_value_at_year_2": value,
    })

future_value_table = pd.DataFrame(future_bond_values)
display(future_value_table.round(4))

```

	state	short_rate_pct	bond_value_at_year_2
0	Low-rate state	1.7355	110.6965
1	Central state	3.2355	104.4224

	state	short_rate_pct	bond_value_at_year_2
2	High-rate state	4.7355	98.5210

```

plot_table = future_value_table.sort_values("short_rate_pct")
bars = plt.bar(plot_table["state"], plot_table["bond_value_at_year_2"],
               color=[BLUE, LIGHT_BLUE, GOLD], edgecolor=INK)
plt.bar_label(bars, fmt="%.2f", padding=3)
plt.ylabel("Bond value at year 2 per 100 face")
plt.title("Future bond value by Hull-White state")
plt.ylim(0, 1.14 * plot_table["bond_value_at_year_2"].max())
plt.show()

```

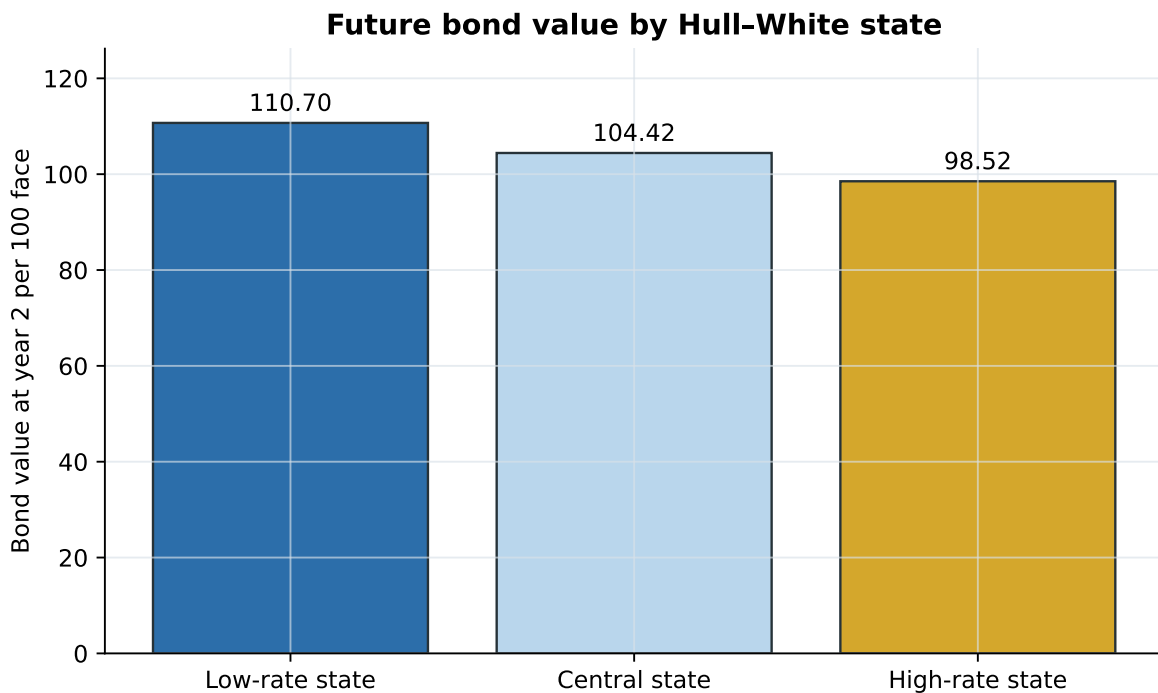


Figure 5: The same promised cash flows have different values in different year-2 rate states

This future valuation is where a short-rate model becomes useful. Today we do not observe $P(2, T)$; Hull-White generates it consistently for each possible r_2 .

10 Step 8: Plain-bond consistency at time 0

Finally, verify the lecture's important claim: once Hull-White is calibrated to the observed curve, it does **not** create a different time-0 price for a vanilla bond. Both methods discount the same fixed cash flows with the same $P(0, T)$.

```
vanilla_payment_times = np.arange(1.0, 11.0)
vanilla_cash_flows = np.full(10, 4.10)
vanilla_cash_flows[-1] += 100.0

# Method 1: discount directly with today's observed/bootstrapped curve.
direct_curve_price = float(np.sum(
    vanilla_cash_flows * discount_0(vanilla_payment_times)
))

# Method 2: use the calibrated Hull-White zero-coupon formula at t = 0.
hw_curve_price = float(np.sum(
    vanilla_cash_flows
    * hw_discount_factor(
        0.0,
        vanilla_payment_times,
        r_0,
        calibrated_a,
        calibrated_sigma,
    )
))

print(f"Direct observed-curve price: {direct_curve_price:.10f}")
print(f"Calibrated Hull-White price: {hw_curve_price:.10f}")
print(f"Difference: {hw_curve_price - direct_curve_price:.12f}")
```

```
Direct observed-curve price: 105.3016516783
Calibrated Hull-White price: 105.3016516783
Difference: 0.000000000000
```

The equality is structural, not a coincidence. At time 0, the calibrated model uses the market discount factors by construction. For a vanilla bond, the model adds no new pricing information. Its value is the ability to generate future curves and to value cash flows that depend on those future states.

11 Checks and interpretation

The workflow has three separate fits, each with a different purpose:

Calibration object	Observable input	What is fitted	Why it matters
Today's discount curve	Coupon-bond prices	$P(0, T)$	Prices fixed cash flows today
Initial Hull-White term structure	Bootstrapped $P(0, T)$	$\theta(t)$	Makes the model no-arbitrage and curve-consistent
Future dynamics	Interest-rate option premiums	a, σ	Controls mean reversion and uncertainty

```
checks = {
    "Bootstrap reprices input bonds": np.max(np.abs(repriced_bonds - clean_prices)) < 1e-10,
    "Discount factors are positive": np.all(bootstrapped_discounts > 0),
    "Discount factors decrease with maturity": np.all(np.diff(bootstrapped_discounts) < 0),
    "Hull-White matches today's discount curve": np.max(
        np.abs(hw_discounts_today - bootstrapped_discounts)
    ) < 1e-10,
    "Two vanilla-bond pricing methods agree": abs(
        hw_curve_price - direct_curve_price
    ) < 1e-10,
}

display(pd.Series(checks, name="passed").to_frame())
assert all(checks.values())
```

	passed
Bootstrap reprices input bonds	True
Discount factors are positive	True
Discount factors decrease with maturity	True
Hull-White matches today's discount curve	True
Two vanilla-bond pricing methods agree	True

12 Takeaways

- Market bond prices can be converted into discount factors by bootstrapping.

- Hull–White does not fit a whole curve using only two constants. The function $\theta(t)$ fits today’s term structure; a and σ describe future dynamics.
- Option prices provide information about those dynamics that vanilla bond prices do not.
- At a future date, each one-factor Hull–White short-rate state produces a complete yield curve and a value for the remaining cash flows.
- A vanilla bond has the same time-0 value under calibrated Hull–White and direct curve discounting. The model earns its keep when future rates affect valuation or cash flows.

i Simplifications

Real desks use multiple instrument types, day-count conventions, accrued interest, bid/ask spreads, smooth curve interpolation, and robust numerical optimizers. The demo deliberately removes those layers so the economic logic stays visible.